

Interview: Allianz Lebensversicherungs-AG

Data Scientist – Fokus Integration Engineering (m/w/d) · Stuttgart

Termin: Montag, 22.09.2026 · Luca Leon Happel

Eckdaten

Team	Leben Advanced Analytics, Bereich Data & Analytics
Auftrag	Datengetriebene Lösungen und KI für Geschäftsprozesse der Lebensversicherung
Vertrag	Unbefristet, Vollzeit, Home-Office möglich („Neues Arbeiten“)
Gehalt	66.000–89.000 € (Stepstone-Schätzung) – eigene Zahl festlegen
Gesucht	Problemlöser:innen, kritisch + kreativ, Ergebnisse an Stakeholder kommunizieren

Die Aufgabe in einem Satz

Schnittstellen bauen zwischen **KI-Komponenten, ABS-Datenbanken und Fachsystemen** – also Lösungsarchitektur, LLM-Integration in bestehende Prozesse, Integrationsmuster und Standards fürs Team, Abstimmung mit OneDIA/AzTech und Architekturdokumentation für Fach- und Führungskräfte.

Das ist keine Modellier-, sondern eine Architektur- und Integrationsrolle. Entsprechend erzählen.

Meine fünf Kernbotschaften

1. **Integration ist mein Kerngeschäft.** monti nicht auf der grünen Wiese, sondern an eine gewachsene Klinik-IT mit hochsensiblen Daten angebunden – Architektur, Schnittstellen, Datenmodelle, seit 2022 im Produktivbetrieb.
2. **DORA, ISO 27001, EU-Datenschutz** vier Jahre im Tagesgeschäft verantwortet – dieselbe Regulatorik, die die Allianz bindet. Das hat kaum ein Data Scientist.
3. **Vom Notebook in den Betrieb.** Kompletter MLOps-Zyklus selbst gebaut: Docker/Kubernetes, CI/CD, Monitoring, Modellpflege.
4. **LLMs in bestehende Abläufe eingebettet** (unstrukturierte klinische Texte) – dieselben Verfahren greifen bei Anträgen, Schriftverkehr, Vertragsdaten.
5. **Stakeholder-Kommunikation.** Business Cases geschrieben und vor Gremien vertreten, die 1,84 Mio. € bewilligt haben; nichttechnische Nutzer geschult.

Anforderungen und meine Abdeckung

	Anforderung	Mein Beleg / meine Antwort
✓	Mind. 2 Jahre Systemintegration, IT-Architektur, Entwicklung im Daten-/KI-Umfeld	4 Jahre IT-Architekt und Lead Dev von monti (UKD), danach Data Scientist bei Corsano
✓	Gewachsene IT-Landschaften, neue Komponenten integrieren	monti an bestehende Klinik-IT angebunden, seit 2022 produktiv
✓	Python, SQL, Datenbankarchitekturen	pandas, scikit-learn, PyTorch; mehrjährige SQL-Praxis und Datenmodellierung
✓	CI/CD, Deployment, Containerisierung	Docker/Kubernetes, CI/CD, Monitoring im Produktivbetrieb
✓	Lösungsarchitektur	Gesamtarchitektur von monti verantwortet, inkl. Schnittstellenschnitt
✓	Master/Bachelor Informatik oder vergleichbar	M.Sc. Mathematik (2026), B.Sc. Informatik mit Data-Science-Schwerpunkt
✓	Deutsch und Englisch	Muttersprache / fließend
!	REST APIs, Messaging, Event-Driven Architecture	REST sicher (u. a. servant); für Eventing ein konkretes monti-Beispiel nennen, nicht abstrakt bleiben
!	LLM-Integration (RAG, LangChain, Bedrock)	LLM-Auswertung klinischer Texte; RAG sauber erklären können, Bedrock als Zugangsweg kennen
!	Terraform / Infrastructure as Code	Nix/NixOS seit Jahren deklarativ – Prinzip identisch, Terraform-Syntax ist Einarbeitung
!	AWS-Toolstack (S3, Lambda, Step Functions, EventBridge, SageMaker)	Keine Produktivpraxis. Muster auf Container/K8s abbilden – Mapping-Tabelle unten sitzt
✗	ABS-Datenbanksysteme und Datenstrukturen	Offen sagen. Brücke: DB2 ist SQL, COBOL-Copybooks kann ich lesen (eigenes COBOL-Projekt)

	Anforderung	Mein Beleg / meine Antwort
x	Kernprozesse der Versicherung	Offen sagen. Antrag → Policierung → Bestandsführung → Leistungsfall benennen können

Wie ich mit den Lücken umgehe

Nicht überspielen. Beide Male: offen zugeben, dann sofort den Weg zeigen, wie ich das schließe, mit Beleg aus der Vergangenheit (libfprint-Treiber, Sicherheitsforschung, COBOL-Projekt: ich arbeite mich regelmäßig in fremde, gewachsene Codebasen ein). Gespieltes Wissen fällt bei Architekten sofort auf und kostet die Stelle.

ABS – Allianz Business System

Was es ist: das selbst entwickelte Kernsystem der Allianz Deutschland für Vertrags- und Schadenbearbeitung (Bestandsführung), quer über Leben, Sach und Kranken. Keine Datenbank – „ABS-Datenbank“ meint die darunterliegende **DB2**.

Zahlen und Geschichte

- über **30 Mio. Verträge** in Deutschland, ca. 42 Mio. in 15 Ländern
- mehrere **Petabyte**, eine der größten Linux-DB2-Installationen überhaupt
- **ca. 300 Schnittstellen** zu Portalen, Vertrieb, Backoffice
- Anwendungslogik in **COBOL**, ursprünglich IBM-Mainframe (z/OS)
- Projekt „**ABS goes Linux**“: entschieden Mitte 2019, Cutover Ostern 2022, komplette DB2 in 48 Stunden migriert, rund 500 Personen Kernteam, verantwortet von Allianz Technology
- seit 2018 auch an andere Versicherer vermarktet (Frankfurter Leben, VBLU)

Was das für die Rolle heißt

- Zugriff über **SQL auf DB2** – SQL überträgt sich, der Dialekt ist Detail
- Strukturen sind **COBOL-geprägt**: breite Copybook-Tabellen, kryptische Feldnamen, gepackte Dezimalfelder (COMP-3), Datum als Zahl, fachliche statt technischer Schlüssel, kaum Constraints, Historisierung über Gültigkeitszeiträume
- Ein Kernsystem dieser Größe verträgt **keine analytischen Direktzugriffe**: der Weg geht über CDC/Replikation oder Batch-Extrakte – dort sitzt OneDIA
- Der **Rückschreibepfad** ins ABS ist fachlich und regulatorisch der schwierige Teil

OneDIA und AzTech

OneDIA

„Das Zuhause für Daten bei Allianz Deutschland“: das zentrale Datenökosystem und die Analytics-Plattform, von BI über Big Data bis KI. Über **200 Datenprodukte** werden gerade vom Legacy-Stack auf **Snowflake und AWS** migriert; Denken in Datenprodukten, data-mesh-nah.

Für mich: Laufzeitumgebung und Datenlieferant zugleich. „Abstimmung mit OneDIA“ heißt: welche Services darf ich nutzen, welche Datenprodukte gibt es schon, wie kommen ABS-Daten dorthin, welche Standards gelten für neue Schnittstellen.

AzTech = Allianz Technology SE

Der konzernweite IT-Dienstleister (früher AMOS), Sitz München, über 13.000 Mitarbeitende in mehr als 20 Ländern. Betreibt Infrastruktur, Anwendungen und Plattformen für alle Allianz-Gesellschaften und hat „ABS goes Linux“ durchgezogen.

Für mich: liefert Plattform, Infrastruktur und Architekturstandards, wir sind Konsument. Konkret: AWS-Landing-Zone und erlaubte Services, Security- und Architektur-Reviews, Freigabeprozesse. Man verhandelt, was on-platform geht und was das Team selbst baut.

Integrationsmuster – das Vokabular der Rolle

Grundentscheidung: synchron (REST/gRPC, Request-Reply) oder asynchron (Messaging, Events, Publish-Subscribe, Queue). Faustregel: fachliche Antwort sofort nötig → synchron; Entkopplung, Lastspitzen, mehrere Abnehmer, langlaufende KI-Jobs → asynchron.

Für gewachsene Landschaften (Kern dieser Stelle)

- **Anti-Corruption Layer** – ABS-Datenmodell schlägt nicht ins KI-Zielmodell durch
- **Strangler Fig** – Altes schrittweise umschließen statt Big Bang
- **API Gateway / Facade** – ein Eingang statt 300 Punkt-zu-Punkt-Verbindungen
- **Change Data Capture** – Änderungen aus DB2 abgreifen statt pollen (Debezium, Q Replication)

Zuverlässigkeit

At-least-once plus Idempotenz · Retry mit exponentiellem Backoff · DLQ und Poison-Message-Handling · Circuit Breaker · Bulkhead · Timeouts · Backpressure. **Merksatz:** „Exactly once gibt es praktisch nicht, nur effectively once durch Idempotenz.“

Governance = „Standards definieren“

API-First mit **OpenAPI-Contract** · semantische Versionierung und Rückwärtskompatibilität · **Schema Registry** und Schema-Evolution · Consumer-Driven

- **Outbox Pattern** – DB-Commit und Event-Versand konsistent, ohne verteiltes 2PC
- **Saga** – langlaufende Transaktionen über Systemgrenzen mit Kompensation
- **Data Products** – versioniert, dokumentiert, mit Owner statt Ad-hoc-Extrakt
- **Batch- und Nachtfenster** – bei Kernsystemen weiter Realität
- kennen, aber nicht überverkaufen: Event Sourcing, CQRS

Klassiker (Hohpe/Woolf)

Message Channel · Content-Based Router · Message Translator · Content Enricher · Splitter/Aggregator · Scatter-Gather · Correlation ID · **Dead Letter Channel** · **Idempotent Receiver** · **Claim Check** (großer Payload nach S3, nur Referenz ins Event)

Muster → AWS-Dienst

Muster	Dienst
Objektspeicher, Claim Check	S3
Kleine Transformation, Event-Handler	Lambda
Orchestrierung, Saga, Retry-Logik	Step Functions
Pub/Sub-Routing, Event-Bus	EventBridge
Queue mit Dead Letter Queue	SQS
Streaming	Kinesis / MSK (Kafka)
ETL	Glue
Modelltraining, Serving-Endpoint	SageMaker
LLM-Zugang	Bedrock
Infrastruktur deklarativ	Terraform (bei mir: Nix)
Analytics-Ebene bei OneDIA	Snowflake

Contract Tests (Pact) · OAuth2/mTLS · Rate Limiting · durchgehende Correlation-ID, Tracing, SLOs · Templates und Referenzimplementierung statt PDF-Richtlinie.

KI- und LLM-Muster

Modell hinter stabiler Inferenz-API (austauschbar) · synchroner Einzelfall vs. asynchrones Batch-Scoring · Feature Store gegen Training/Serving-Skew · **RAG** mit Pflicht zur Quellenangabe statt Fine-Tuning · Guardrails und Output-Validierung gegen Schema · Human-in-the-Loop mit gespeicherter Korrektur · Prompt-, Modell- und Indexversionierung · Kosten- und Latenzbudget · Fallback bei Modellausfall · **Audit-Tail** (Input, Modellversion, Output, Entscheider) – Pflicht wegen DORA und AI Act.

Regulatorik – mein Heimvorteil

- **DORA** gilt für Versicherer direkt: IKT-Risikomanagement, Vorfallmeldung, Resilienztests, Drittparteienrisiko – letzteres greift, sobald ein externer LLM-Anbieter im Spiel ist. Vier Jahre selbst verantwortet.
- **EU AI Act**: Risikobewertung und Preisbildung in der Lebens- und Krankenversicherung ist explizit **Hochrisiko** (Anhang III). Für Unterstützungsprozesse wie Dokumentenauswertung gilt das nicht automatisch – diese Unterscheidung zu kennen, wirkt.
- **VAIT/BaFin**, Auslagerungsmanagement, Nachvollziehbarkeit von Modellentscheidungen gegenüber Kunden und Aufsicht.
- **ISO 27001** und EU-Datenschutz für hochsensible Daten: Tagesgeschäft bei monti.

Wahrscheinliche Fragen und meine Stichworte

Frage	Antwortpfad
„Erzählen Sie von monti.“	Problem → Architektur (Dienste, Datenflüsse, Schnittstellen) → Integration in Bestandssysteme → Betrieb und Monitoring → Ergebnis. Eine konkrete Zahl und eine Anekdote parat haben.
„Wie integrieren Sie ein LLM in einen bestehenden Prozess?“	Use Case eingrenzen → Datenzugang und Datenschutz → RAG statt Fine-Tuning, mit Quellenangabe → Schnittstelle (REST synchron oder Event/Batch) → Evaluation → Guardrails und Monitoring → Human-in-the-Loop → Rollout. Beispiel: klinische Texte.
„Sie kennen ABS nicht.“	Stimmt. DB2 ist SQL, Copybook-Strukturen kann ich lesen, gewachsene fremde Codebasen sind mein Alltag. Plan: erste Wochen Datenmodell und Schnittstellenkatalog systematisch durcharbeiten, dann Integrationsmuster darauf festlegen.
„AWS-Erfahrung?“	Keine Produktivpraxis, offen gesagt. Die Muster kenne ich und habe sie containerbasiert selbst betrieben – Mapping-Tabelle oben. Einarbeitung ist Handwerk, kein Umdenken.
„Event-Driven oder REST – wann was?“	Synchron für sofortige fachliche Antwort, asynchron für Entkopplung, Lastspitzen, mehrere Abnehmer, lange Laufzeiten. Dazu Idempotenz, Retries, DLQ, Schema-Evolution.
„Wie definieren Sie Standards fürs Team?“	Bestand analysieren → wenige verbindliche Muster festlegen (API-First, OpenAPI-Contract, Versionierung, Auth, Logging) → Templates und Referenzimplementierung → leichter Review-Prozess. Standards, die niemand benutzt, sind keine.

Frage	Antwortpfad
„Warum Versicherung, warum Allianz Leben?“	Regulierter Bereich mit sensiblen Daten kenne ich aus der Medizin und mag ich. Lebensversicherung heißt langlaufende Verträge und riesige Bestandsdaten – hoher Hebel. Das Team baut Advanced Analytics gerade auf, da will ich mitgestalten.
„Warum Corsano nach vier Monaten?“	Vorher festlegen und kurz halten. Keine Kritik am Arbeitgeber, sachlicher Grund, Blick nach vorn.
„Gehaltsvorstellung?“	Zahl vorher festlegen. Orientierung: 66.000–89.000 €, Tarif Versicherungswirtschaft.
„Umzug nach Stuttgart?“	Ja, gerne. Hybrides Arbeiten willkommen.

Meine Fragen ans Team

- Greifen die KI-Komponenten direkt auf ABS zu oder über replizierte Datenprodukte in OneDIA? Und gibt es Schreibpfade zurück ins ABS?
- Welche KI- und LLM-Use-Cases laufen bereits produktiv, welche stehen als Nächstes an?
- Wie ist die Zusammenarbeit mit OneDIA und AzTech organisiert – wer entscheidet über Plattform und Architekturstandards?
- Wie sieht der Deployment-Weg aus: AWS-Landing-Zone, Freigaben, DORA-Anforderungen?
- Teamgröße und Rollen – wer hat Integration Engineering bisher gemacht?
- Wie ist das Verhältnis von Konzeption zu eigener Umsetzung in dieser Rolle?
- Wie viel Präsenz in Stuttgart erwartet das Team?
- Wie geht der Prozess nach diesem Gespräch weiter?

Vor dem Gespräch abhaken

- ☐ Gehaltszahl festgelegt
- ☐ Corsano-Antwort in zwei Sätzen formuliert
- ☐ monti-Architektur als Skizze aufmalbar (Whiteboard-tauglich)
- ☐ Kernprozesse Leben: Antrag, Underwriting, Policierung, Bestandsführung, Leistungsfall, Inkasso, Rückkauf, Überschussbeteiligung
- ☐ Lebenslauf und Motivationsschreiben ausgedruckt dabei
- ☐ Format, Uhrzeit, Teilnehmer, Dauer bestätigt
- ☐ COBOL-Projekt und Publikation (European Heart Journal – Digital Health) als Referenz parat
- ☐ Zwei eigene Fragen, die ich auf jeden Fall stellen will, markiert